In-Memory Implementation of an Approximate Adder With Reduced Latency and Error

Vijaya Lakshmi, Vikramkumar Pudi, *Member, IEEE*, and John Reuben

***Abstract*— In-memory computing has been a prominent solution to Von Neumann bottleneck that degrades the performance of a computing system. Approximate computing is widely used to improve the performance of multimedia and other applications that are error-tolerant. Approximate adders being the basic units used to design other complex units, get benefited when implemented in-memory by taking the advantages of both in-memory computing and approximate computing. In this work, we have improved the speculative carry select adder to minimize error and critical path delay by eliminating multiplexers. The proposed adder achieves less critical path, area, improved error characteristics such as error rate, normalized mean error distance and mean relative error distance when compared to the state-of-the-art approximate adders. Error rate of the proposed adder is 34.48% less than the best reported 32-bit adder with sub-adder size of 8-bit. When the proposed approximate adders are implemented in-memory using majority logic, they achieve better performance compared to the existing in-memory approximate adders. Latency of the proposed adders is observed to be a constant irrespective of adder size for a fixed sub-adder size.**

***Index Terms*— In-memory computing, approximate computing, 1T–1R array, resistive random access memory, speculative adder, error-tolerant.**

I. Introduction

IN-MEMORY computing (IMC) has been a promising solution to memory wall problem in the Von Neumann architectures [1], [2], [3], [4]. The need for the IMC is evident from a quantitative example in [5], where it is given that the energy for the cache memory access (10pJ-100pJ) is several orders of magnitude higher than the computation energy for arithmetic operations (0.03pJ-3.7pJ) in 45nm. IMC using emerging non-volatile technologies such as ReRAM increases the integration density and energy efficiency by incorporating the computational capabilities in a memory array. Approximate computing has been another technique to minimize the energy consumption in the recent years [6], [7], [8], [9]. The simultaneous improvement in energy efficiency, performance, and density of a computing system is always challenging as optimization of one affects the others. The compromise in quality with approximate computing can improve either of these metrics. The integration of these two techniques, *i.e* implementation of approximate arithmetic circuits in-memory can simultaneously improve these metrics as required in processing the ever-increasing information amount in the current applications. The target of approximate computing is toward the error-tolerant applications such as multimedia processing that accepts approximate outputs instead of precise outputs due to the limited human perceptual capability, high-speed cryptographic applications [10] where the number of erroneous bits matters regardless of their position, accelerate the multiply-and-accumulate operation in emerging applications such as neural networks [11], etc. Addition is the basic operation that is most frequently used in these applications. In most of the practical cases, length of the carry chain is significantly less than the size of the adder. The probability for the length of a carry chain being shorter than 5 bits is more than 90% [12].

Approximate adders implemented in-memory using IMPLY logic [13] have high latency. The IMPLY logic requires external resistors (R_G), switches, drivers, different operating voltages (V_{Cond} and V_{Set}) and an isolation voltage to the non operating cells to reduce the sneak-path effects. These requirements complicate the peripheral circuitry. A library of approximate adders (referred to as "IMAGIN") for in-memory computing based on MAGIC-NOR and IMPLY logic is introduced in [14]. The library IMAGIN allows a designer to select among the different approximate adders with variable error metrics. The approximate adder designs reported with less latency from this library have large errors. Implementation of MAGIC-NOR requires a special ReRAM with asymmetrical switching voltages (V_{SET} and V_{RESET}), special execution voltage and isolation voltage to avoid NOR operation in rows/columns where it is not intended. Approximate adders design using the memristor ratioed logic (MRL) has been proposed in [15]. The area and number of cycles required for execution using MRL are high. It is not sneak-path free as it uses a passive crossbar array and there exists significant design effort. A full adder implementation using MRL requires 18 memristors and 8 transistors [16]. The performance and error characteristics of the approximate adder can be improved and also the sneak-path effects can be mitigated using majority logic which can be efficiently implemented in a transistor accessed ReRAM array. Majority logic is widely used to implement a Boolean logic in emerging technologies [17],

Received 7 July 2024; revised 1 October 2024, 1 December 2024, and 2 December 2024; accepted 2 December 2024. Date of publication 18 December 2024; date of current version 30 April 2025. This article was recommended by Associate Editor X. Fong. *(Corresponding author: Vikramkumar Pudi.)*

Vijaya Lakshmi and Vikramkumar Pudi are with the Department of Electrical Engineering, Indian Institute of Technology at Tirupati, Tirupati 517506, India (e-mail: ee20d001@iittp.ac.in; vikram@iittp.ac.in).

John Reuben is with the Chair of Computer Architecture, Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU), 91058 Erlangen, Germany (e-mail: johnreuben.prabahar@fau.de).

Digital Object Identifier 10.1109/TCSI.2024.3511955

1549-8328 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:33:55 UTC from IEEE Xplore. Restrictions apply.

[18]. A three-input majority operation has been realized as a read operation in a 1T–1R array [19] using a time-based sense amplifier. It demands less peripheral circuitry modifications as the conventional memory array already supports the read operation. The number of cycles required for implementing a full adder in-memory is 17, 10, 6 cycles using IMPLY [20], MAGIC-NOR [21] and MAJORITY [22] logic respectively.

Speculative carry generation is the widely used approach in the carry select adders, generation of conditional sum (one sum with carry-in '0' and other sum with carry-in '1'), floating point rounding [23], etc. Speculative Carry Select Adder (SCSA) is the fastest approximate adder among all the existing approximate adders with good error characteristics but the area overhead is more due to the multiplexers used for carry selection [24]. In this work, we propose a majority logic based SCSA that completely eliminates multiplexers and has reduced error rate, critical path and area compared to the state-of-the-art approximate adders. Majority gates are used for carry selection purposes instead of multiplexers. This is more advantageous for in-memory implementation where a three-input majority gate can be realized as a single read operation, whereas a multiplexer requires at least 3 majority gates (explained in Section III).

The remaining part of this paper is organized as follows: The basics of approximate adders, ReRAM and majority logic realization are explained briefly in Section II. The proposed majority logic based SCSA is explained in Section III. The in-memory implementation of the proposed adder and its comparison with the state-of-the-art are detailed in Section IV. Proposed approximate adders are compared with the state-of-the-art in terms of latency, area, error characteristics and complexity is analysed in Section V. Section VI concludes the paper. The contributions of this work are as follows:

- A majority-based speculative carry select adder design with reduced critical path, area, and improved error characteristics.
- An in-memory implementation of the proposed approximate adder which has reduced latency compared to the state-of-the art in-memory approximate adders.

II. Background

This Section provides the necessary basics to implement approximate circuits in-memory using majority logic: Resistive random access memory, majority logic realization in a 1T–1R array, quality metrics, and types of approximate adders.

A. Resistive Random Access Memory (ReRAM)

ReRAM is an emerging non-volatile memory technology that belongs to memristor family, known for its high density, low WRITE energy, good scalability, resistance window, and low WRITE energy [25]. Application of a suitable voltage across the ReRAM cell results in a change of its resistance state from low (LRS) to high resistance states (HRS) and vice-versa [26]. A transistor in series with ReRAM is considered as a 1-bit memory cell (1T–1R cell) which consists of three terminals: bit-line (BL), word-line (WL) and source-line (SL).

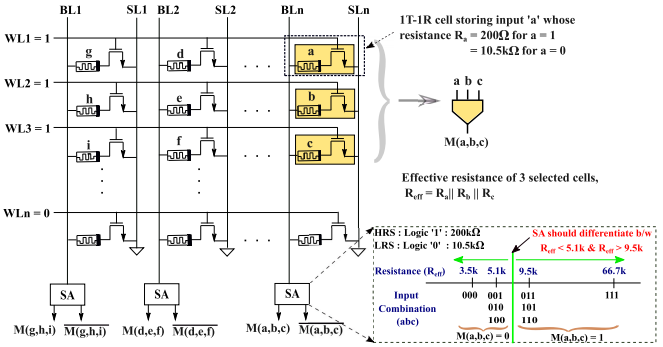


Fig. 1. A three-input majority logic realization in a 1T–1R array. The select cells for the majority operation are highlighted. WLs of the selected cells are made high and all other WLs are grounded.

Word-line is connected to the gate of the transistor, bit-line is connected to the top-electrode of the ReRAM and source-line is connected to the source of the transistor. A memory array constructed by arranging several such 1T–1R cells in rows and columns is referred to as 1T–1R array (see Fig. 1).

B. Majority Logic Realization in a 1T–1R Array

Majority logic is a Boolean logic in which the output is defined to be true if more than half of the n inputs are true, where n is odd. Majority logic can be implemented in ReRAM array as a READ operation *i.e.* by using a sense amplifier connected to the column. Having a sense amplifier (SA) [27] connected to the column, reading a column (BL) of a 1T–1R array by selecting three rows (WLs) simultaneously results in the three-input majority operation of the contents stored in the selected cells [19]. In Fig. 1, a, b and c are the inputs stored in the form of a resistance (LRS/HRS corresponds to logic '0'/'1') in the selected three cells of 1st column and the result of majority operation is $M(a, b, c) = ab + bc + ac$. Here the effective resistance of the selected cells $R_{eff} = R_a||R_b||R_c$ (R_a, R_b, R_c are the resistances of the cells storing the inputs a, b and c respectively). Assuming a LRS of 10.5 kΩ and a HRS of 200 kΩ as illustrated in Fig. 1, when two out of the three cells are in LRS, the effective resistance R_{eff} is < 5.1kΩ and when two out of the three cells are in HRS, R_{eff} is $>$ 9.5kΩ. A sense amplifier [22] which can differentiate between these two cases will implement a majority logic operation during READ operation. The SA outputs both the majority operation result and its complement thereby avoiding the need for an extra inverter when implementing circuits. Having a dedicated SA for each column of the array, multiple such majority operations can be performed in parallel ($M(a, b, c)$, $M(d, e, f)$ $\ldots M(g, h, i)$ are computed in parallel in Fig. 1) resulting in latency optimized designs. Any arithmetic circuit can be implemented as a sequence of majority, read and write operations. Circuits such as adder, multiplier, inner product and comparator [22], [27], [28], [29] are effectively implemented in a 1T–1R array using majority logic. During the computation, the intermediate results produced at the SA output in one step can be written back to the array and used as inputs for the next step operations. This concept of computing majority by activating three WLs and sensing the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:33:55 UTC from IEEE Xplore. Restrictions apply.

current corresponding to R_{eff} has been experimentally verified in 1T-1R array in [30].

C. Quality Metrics for Approximate Adders

Approximate computing requires evaluation of accuracy in addition to the conventional figure of merits such as power, delay and area. If Y_{acc}, Y_{apx} are the accurate and approximate outputs respectively and X is the total number of input combinations, quality metrics used to measure the accuracy of the approximate circuits are described as follows:

1) Error Distance (ED): Modulus of the difference between the Approximate and accurate outputs.

$$ED = |Y_{apx} - Y_{acc}| \tag{1}$$

2) Error Rate (ER): Percentage of incorrect outputs out of all outputs for different input combinations. If $P(ED_i \neq 0)$ is the probability of $ED_i \neq 0$,

$$ER = \frac{1}{X}\sum_{i=0}^{X-1} P(ED_i \neq 0) \tag{2}$$

3) Mean Error Distance (MED): Average error distance for a given input set.

$$MED = \frac{1}{X}\sum_{i=0}^{X-1} ED_i \tag{3}$$

4) Mean Relative Error Distance (MRED):

$$MRED = \frac{1}{X}\sum_{i=0}^{X-1} \frac{|Y_{apx} - Y_{acc}|}{Y_{acc}} \tag{4}$$

5) Normalized Mean Error Distance (NMED): Normalized value of the MED by the maximum accurate adder output.

$$NMED = \frac{MED}{2^n} \tag{5}$$

6) Mean Squared Error (MSE): Mean of the squares of the all error distances.

$$MSE = \frac{1}{X}\sum_{i=0}^{X-1} ED_i^2 \tag{6}$$

D. Types of Approximate Adders

There exist various types of approximate adders in the literature [31] that reduce the complexity and critical path delay of an accurate adder. This section briefly explains the different types of approximate adders.

1) Almost Correct Adder (ACA): An n-bit ACA uses k ($n > k$) least significant bits (LSBs) for carry prediction at each sum bit. The critical path delay for this adder is $O(k)$ if ripple carry implementation is used and it reduces to $O(log(k))$ if parallel implementation such as parallel-prefix, carry-look-ahead are used. The area overhead is high as an n-bit ACA requires $(n-k)$ sub-adders of k-bit each.

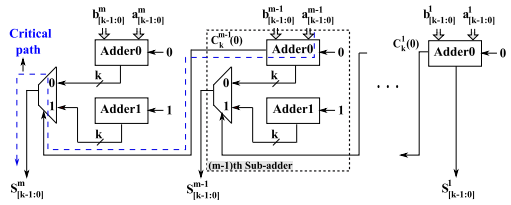


Fig. 2. Block diagram of a conventional Speculative carry select adder. A, B are the inputs, S is the output, **Adder0** and **Adder1** are k-bit adders with carry-in '0' and carry-in '1' respectively. $a^m_{[k-1:0]}$ and $b^m_{[k-1:0]}$ denote the k-bit inputs of the mth sub-adder.

2) Segmented Adders: This type of adders divide an n-bit adder into smaller k-bit segments ($\lceil \frac{n}{k} \rceil$ number of sub-adders) that operate in parallel. An equal segmentation adder (ESA) of this type uses fixed inputs for each sub-adder. The delay is same as ACA and the hardware complexity is less. Error tolerant adder (ETA) of this type uses k-bit sum and carry generators in each sub-adder. The **Sum generator** in each sub-adder uses the carry generated from the previous carry generator. The hardware complexity is same and the accuracy is more compared to ESA. However, the delay is increased to $O(log(2k))$.

3) Carry Select Adders: These adders divide an n-bit adder into $m = \lceil \frac{n}{k} \rceil$ sub-adders of k-bit that operate in parallel. The Speculative carry select adder (SCSA) [32] of this type uses two k-bit adders (one adder with carry-in '0' and other with carry-in '1') in each sub-adder as shown in Fig. 2. Inputs to the mth sub-adder are denoted as $a^m_{[k-1:0]} = a_{mk-1:mk-k}$ and $b^m_{[k-1:0]} = b_{mk-1:mk-k}$ (m varies from 1 to $\lceil \frac{n}{k} \rceil$). The output of mth sub-adder is the result of **Adder0** or **Adder1** depending on the carry produced by **Adder0** from $(m-1)^{th}$ sub-adder. This adder is approximate as it uses k-bits instead of n-bits for carry generation. The accuracy is same as ETA but the circuit complexity, power consumption and the critical path delay are high. Carry speculative adder (CSPA) [33] uses a carry predictor to select the carry signals for the **Sum generator** in each sub-adder. The hardware complexity is reduced compared to SCSA but the critical path delay is high.

A High Accuracy Block-based Approximate adder (HABA) in [35] uses generate signals for speculation of carry and also the error correction unit. The error rate of HABA is low compared to all other approximate adders but the critical path delay is still high due to the rippling of carry and multiplexers. A Block-based Carry Speculative Approximate adder (BCSA) reported in [36] has high error rate and less critical path delay than the HABA. In the worst case, the critical path of a BCSA consists of two sub-adders and a multiplexer. Both BCSA and HABA use some additional circuitry in each sub-adder for error correction. The HABA [35] and BCSA [36] are not the in-memory implementations. An additional error recovery and reduction unit is used to improve the accuracy. The approximate adder design (DSLLA) in [37] splits an n-bit adder into two sub-adders (accurate and approximate) and again divides the accurate sub-adder into upper and lower sub-adders to reduce the length of the carry chain. The approximate adders design in [38] use majority logic based approximate

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:33:55 UTC from IEEE Xplore. Restrictions apply.

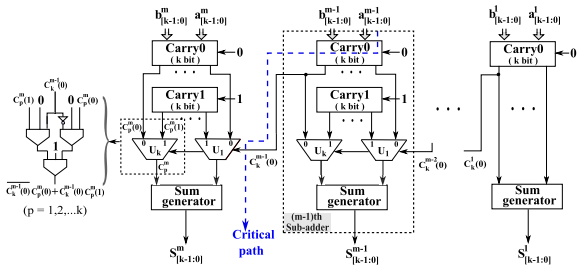


Fig. 3. Block diagram of an n-bit SCSA using k 2:1 multiplexers (U_1-U_k) for the carry selection in each sub-adder [34]. A 2:1 multiplexer implementation requires 3 majority gates (left), resulting in more latency and area. $a^m_{[k-1:0]} = a_{mk-1:mk-k}$ and $b^m_{[k-1:0]} = b_{mk-1:mk-k}$ denote the k-bit inputs of the mth sub-adder.

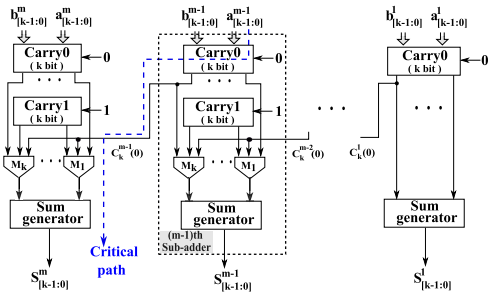


Fig. 4. Block diagram of the proposed n-bit SCSA-1. k majority gates ($M1$-Mk) are used instead of k 2:1 multiplexers for the carry selection in a sub-adder thereby reducing the area and latency. $a^m_{[k-1:0]}$ and $b^m_{[k-1:0]}$ denote the k-bit inputs of the mth sub-adder.

full adders. These adders are not implemented in-memory and their accuracy could be improved further.

Approximate adders reported in [39] are based on modification (rearranging and deleting sub-blocks) of parallel prefix adders structure for performance improvement. Approximate multi bit full adders and compressors are designed in [40]. Bit-truncation approach is used for approximate adders in [41] where the input bits to be truncated are set to the constant value. The approximate adder reported for low error rate in the literature has high critical path delay and the approximate adder reported for low critical path delay has high error rate. The multiplexers used in these adders require more cycles and area when implemented in-memory. In this work, we propose an approximate adder that uses majority gates instead of multiplexers for the carry selection, suitable for in-memory implementation and has better error characteristics, less critical path than the best reported approximate adders in the literature.

III. Proposed Majority Logic Based SCSA

This section explains the proposed multiplexer-less SCSA using majority logic. The SCSA shown in Fig. 3 [34] is another version of the conventional SCSA in Fig. 2. The n-bit SCSA in Fig. 3 consists of $\lceil \frac{n}{k} \rceil$ sub-adders and the sub-adder consists of two carry generation blocks (**Carry0** and **Carry1**) and a **Sum generator** instead of two adders in the conventional SCSA (in Fig. 2). A **Carry0** block takes '0' as the carry input and a **Carry1** block takes '1' as the carry input. The area is less compared to the SCSA in Fig. 2. However, the critical path still remain high due to the multiplexers used for the carry selection. We realize the entire adder in terms of majority gates and then map the resulting circuit to a 1T–1R array. Each 2:1 multiplexer requires three majority gates in general, which makes the design complex in terms of area and delay.

In this work, we use a three-input majority gate instead of a 2:1 multiplexer for the carry selection in a sub-adder and implement the entire adder purely using majority gates. The proposed majority logic-based SCSA-1 (in Fig. 4) completely eliminates the multiplexers in the critical path of conventional SCSA without affecting its functionality, thereby reducing the area, delay, and energy consumption when implemented in-memory. We further improve the accuracy of the proposed SCSA-1 by slightly modifying the carry select input to the most significant (m^{th}) sub-adder, named as the proposed SCSA-2 (in Fig. 7). A single majority gate can produce the carry-out of three inputs a, b and c as $M(a, b, c) = ab + bc + ac$. Let $A = a_{n-1}a_{n-2}\ldots a_1a_0$, $B = b_{n-1}b_{n-2}\ldots b_1b_0$ and C_{in} (initial carry) be the inputs to an n-bit adder, a recursive majority logic based expression for output carry (C_n) [42] is given by Equation (7),

$$C_n = M(a_{n-1}, b_{n-1}, M(a_{n-2}, b_{n-2}, \ldots, M(a_1, b_1, M(a_0, b_0, C_{in}))) \quad (7)$$

For the SCSA using multiplexers for carry selection in a sub-adder (in Fig. 3), let $C^m_p(0)$, $C^m_p(1)$ ($p = 1, 2, \ldots, k$) be the carries produced by the carry generators **Carry0** ($C_{in} = 0$) and **Carry1** ($C_{in} = 1$) of the mth sub-adder at $(p-1)^{th}$ bit position respectively.

$$C^m_p(0) = M(a^m_{p-1}, b^m_{p-1}, M(a^m_{p-2}, b^m_{p-2}, \ldots, M(a^m_1, b^m_1, M(a^m_0, b^m_0, 0))) \quad (8)$$

$$C^m_p(1) = M(a^m_{p-1}, b^m_{p-1}, M(a^m_{p-2}, b^m_{p-2}, \ldots, M(a^m_1, b^m_1, M(a^m_0, b^m_0, 1))) \quad (9)$$

If $C^{m-1}_k(0)$ is the carry produced at the most significant bit ($p = k$) of the carry generator **Carry0** of the $(m-1)^{th}$ sub-adder, expressions for the selected carry (C^m_p) and sum output (S^m_{p-1}) [42] at the $(p-1)^{th}$ bit position of an mth sub-adder are given by Equations (10) and (11):

$$C^m_p = \overline{C^{m-1}_k(0)}C^m_p(0) + C^{m-1}_k(0)C^m_p(1) \quad (10)$$

$$S^m_{p-1} = a^m_{p-1} \oplus b^m_{p-1} \oplus C^m_{p-1} = M(M(\overline{C^m_p}, a^m_{p-1}, b^m_{p-1}), \overline{C^m_p}, C^m_{p-1}) \quad (11)$$

A. Proposed Majority Logic Based SCSA-1

The proposed SCSA-1 in Fig. 4 completely eliminates the multiplexers in the critical path of conventional SCSA thereby reducing the delay, area, and power consumption. The carries of **Carry0** and **Carry1** blocks in the proposed SCSA-1 are computed by using the Equations (8), (9). And the selected carry is derived in terms of majority logic using the Property 1 and Property 2 [42]. Let a^m_{p-1} and b^m_{p-1} are the input bits at $(p-1)^{th}$ bit position in m^{th} sub-adder (marked in Fig. 5). From Table I, comparing column 4 and 5, Property 1 is verified. Comparing column 3 and 6, Property 2 is verified.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:33:55 UTC from IEEE Xplore. Restrictions apply.

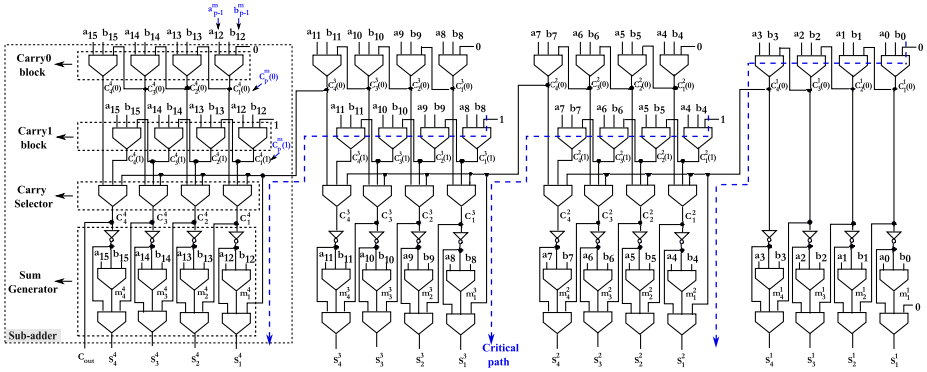


Fig. 5. Majority logic-based diagram of the proposed 16-bit SCSA-1 with sub-adder size $k = 4$. **Carry0** and **Carry1** modules in each sub-adder operate in parallel.

TABLE I

EXAMPLE TO VERIFY ALL POSSIBLE COMBINATIONS IN PROPERTY 1 AND PROPERTY 2. LET X = $C_p^m(0) + C_p^m(1)$ AND Y = $C_p^m(0).C_p^m(1)$

a_{p-1}^m	b_{p-1}^m	$C_p^m(0) = M(a_{p-1}^m, b_{p-1}^m, 0)$	$C_p^m(1) = M(a_{p-1}^m, b_{p-1}^m, 1)$	X	Y
0	0	0	0	0	0
0	1	0	1	1	0
1	0	0	1	1	0
1	1	1	1	1	1

Property 1: $C_p^m(1) = C_p^m(0) + C_p^m(1)$

Property 2: $C_p^m(0) = C_p^m(0).C_p^m(1)$

Theorem 1: Multiplexer expression for the selected carry $C_p^m = \overline{C_k^{m-1}(0)}C_p^m(0) + C_k^{m-1}(0)C_p^m(1)$ can be expressed as a single majority gate operation $C_p^m = M(C_p^m(0), C_p^m(1), C_k^{m-1}(0))$.

Proof: Substitution of Property 1 in Equation (10) results in,

$$\begin{aligned} C_p^m &= \overline{C_k^{m-1}(0)}C_p^m(0) + C_k^{m-1}(0)C_p^m(1) \\ &= \overline{C_k^{m-1}(0)}C_p^m(0) + C_k^{m-1}(0)\left[C_p^m(0) + C_p^m(1)\right] \\ &= \left[\overline{C_k^{m-1}(0)} + C_k^{m-1}(0)\right]C_p^m(0) + C_k^{m-1}(0)C_p^m(1) \\ &= C_p^m(0) + C_k^{m-1}(0)C_p^m(1) \end{aligned} \tag{12}$$

By Substituting Property 1 and 2 in Equation (12)

$$\begin{aligned} C_p^m &= C_p^m(0) + C_k^{m-1}(0)C_p^m(1) \\ &= C_p^m(0) \cdot C_p^m(1) + C_k^{m-1}(0)\left[C_p^m(0) + C_p^m(1)\right] \\ &= C_p^m(0) \cdot C_p^m(1) + C_k^{m-1}(0) \cdot C_p^m(0) + C_k^{m-1}(0) \cdot C_p^m(1) \\ &= M(C_p^m(0), C_p^m(1), C_k^{m-1}(0)) \end{aligned} \tag{13}$$

The Equation (13) is the modified form of Equation (10) where a multiplexer is replaced by a majority gate, which reduces the critical path, area and makes the design suitable for in-memory implementation. The carry generator block can be constructed using ripple carry logic or high-speed prefix method [42]. The critical path of this adder consists of $(k+3)$ majority gates (k majority gates in the carry generator and 3 majority gates in the **Sum generator**) when using ripple carry logic and it is $(log_2(k)+3)$ ($log_2(k)$ majority gates in the carry generator and 3 majority gates in the **Sum generator**) when using a high-speed prefix method [42] for carry generation. For ease of understanding, we have shown the ripple carry logic for carry generation in the diagrams. The majority logic-based diagram of the proposed 16-bit SCSA-1 with sub-adder size of 4-bit is shown in Fig. 5. Here the 16-bit adder is divided into four 4-bit sub-adders ($m = k = 4$). A ripple carry method is used for implementing carry generators **Carry0** and **Carry1**. The critical path consists of seven 3-input majority gates (4 for carry generation and 3 for sum generation). The total number of majority gates required is 72.

Area of the proposed SCSA-1 in Fig. 5 can be optimized by eliminating the two majority gates with the least significant bit inputs in **Carry0** and **Carry1** blocks of each sub-adder by considering the fact that $M(M(a_0, b_0, 0), M(a_0, b_0, 1), C_{in})$ is same as $M(a_0, b_0, C_{in})$. The modified carry generator blocks are named **Carry-a** and **Carry-b**.

The Equations (8) and (9) are redefined as Equations (14) and (15):

$$\begin{aligned} C_p^m(a) = M(&a_{p-1}^m, b_{p-1}^m, M(a_{p-2}^m, b_{p-2}^m, \ldots, \\ &M(a_1^m, b_1^m, a_0^m)) \end{aligned} \tag{14}$$

$$\begin{aligned} C_p^m(b) = M(&a_{p-1}^m, b_{p-1}^m, M(a_{p-2}^m, b_{p-2}^m, \ldots, \\ &M(a_1^m, b_1^m, b_0^m)) \end{aligned} \tag{15}$$

Using Equations (14) and (15) for computing carries in the carry generator blocks, selected carry at the $(p-1)^{th}$ bit position of an mth sub-adder of the proposed n-bit SCSA-1 (area-optimized) (m = 1,2,…,$\lceil \frac{n}{k} \rceil$ and p = 1,2,…,k) is calculated using Equations (16) and (17),

$$C_p^m = M(C_p^m(a), C_p^m(b), C_k^{m-1}(o)) \tag{16}$$

$$C_k^{m-1}(o) = M(C_k^{m-1}(a), C_k^{m-1}(b), 0) \tag{17}$$

The area-optimized design reduces two majority gates in the carry generator blocks and adds one majority gate for output carry generation in each sub-adder except the least significant block. Fig. 6 shows the area-optimized form of the proposed 16-bit SCSA-1 in Fig. 5. If there are m sub-adders in an n-bit adder, area-optimized design of the proposed SCSA-1 uses m majority gates lesser than the proposed SCSA-1. Critical path delay for area-optimized design is the same as the proposed SCSA-1 *i.e.* seven majority gates.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:33:55 UTC from IEEE Xplore. Restrictions apply.

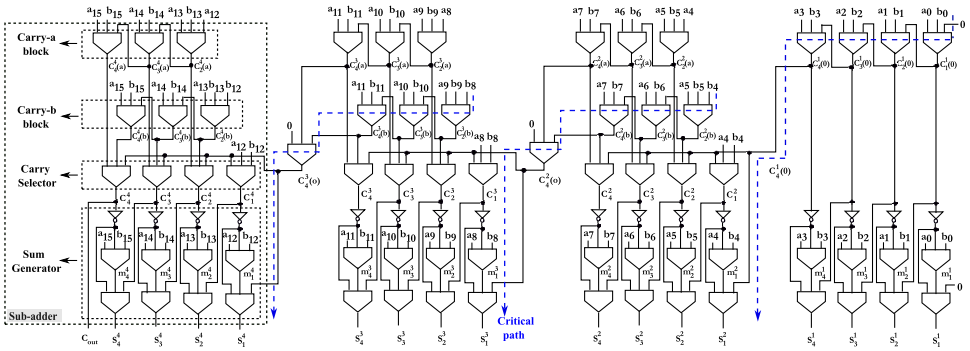


Fig. 6. Majority logic-based diagram of the proposed 16-bit SCSA-1 (area-optimized) with sub-adder size $k = 4$. **Carry-a** and **Carry-b** modules in each sub-adder operate in parallel.

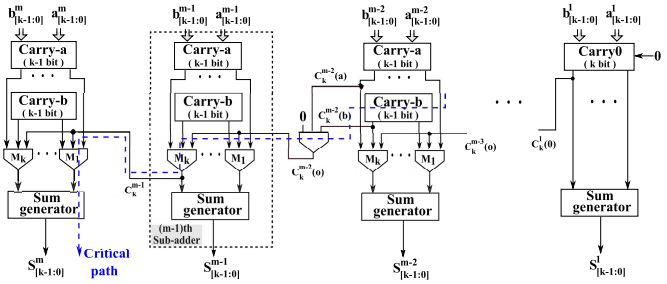


Fig. 7. Block diagram of the proposed n-bit SCSA-2. The carry propagation path to the most significant sub-adder is changed to improve the accuracy. $a^m_{[k-1:0]}$ and $b^m_{[k-1:0]}$ denote the k-bit inputs of the mth sub-adder.

B. Proposed Majority Logic Based SCSA-2

We have calculated the error characteristics such as ED, MED, ER, NMED, MSE, and MRED (see Table III) for the proposed SCSA-1, which are of the same order of other approximate adders. The accuracy of the proposed SCSA-1 (area-optimized) in Section III-A can be improved by slightly modifying the carry select input to the most significant (m^{th}) sub-adder. The modification is made to the most significant sub-adder as it affects error percentage the most. Fig. 7 shows the block diagram of the proposed SCSA-2 derived from the proposed SCSA-1 (area-optimized). The selected carry at k^{th}-bit position (C_k^{m-1}) from the $(m-1)^{th}$ sub-adder is passed to the mth sub-adder instead of the carry $C_k^{m-1}(o)$. The remaining portion is the same as the proposed SCSA-1 (area-optimized). One majority gate is added to the critical path in the proposed SCSA-2 irrespective of the adder size, which will be a latency of two cycles when mapped to a 1T–1R array. Critical path of this adder consists of $k + 4$ and $log_2(k) + 4$ majority gates when using ripple carry logic and high-speed prefix method for carry generation respectively. The majority logic-based diagram of the proposed 16-bit SCSA-2 with sub-adder size of 4-bit is shown in Fig. 9 where the critical path consists of eight majority gates. Equations (14) and (15) are used for computing carries in the carry generator blocks. The selected carry at the $(p-1)^{th}$ bit position of an mth sub-adder ($m = 1,2,\ldots,\lceil \frac{n}{k} \rceil$-1 and $p = 1,2,\ldots,k$) is computed using Equations (16) and (17). The selected carry at the $(p-1)^{th}$ bit position of an $\lceil \frac{n}{k} \rceil$th sub-adder ($p = 1,2,\ldots,k$) is- given by Equation (18),

$$C_p^m = M(C_p^m(a), C_p^m(b), C_k^{m-1}) \tag{18}$$

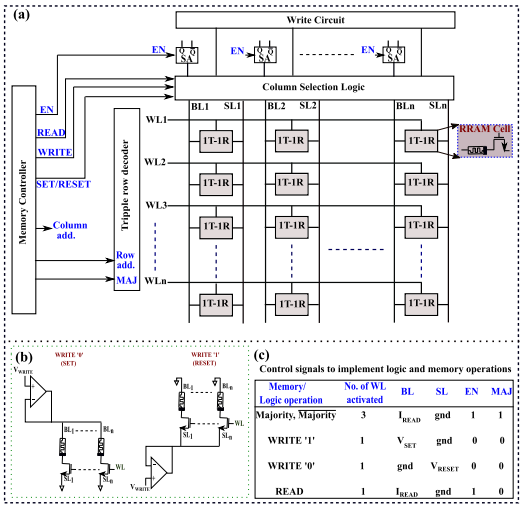


Fig. 8. (a) In-memory computing system architecture [28]. (b) WRITE circuit for parallel write operations into columns of the array. (c) Control signals: MAJ = 1 enables majority operation and MAJ = 0 enables normal READ/WRITE operation.

IV. In-Memory Implementation of the Proposed SCSA

This section explains the implementation of the proposed majority logic-based SCSA in a 1T–1R array. The proposed adders are implemented as a sequence of READ (Majority) and WRITE operations. Fig. 8 shows the architecture of an in-memory computing system. We follow the method used for a three-input majority gate realization in a 1T–1R array (as shown in Fig. 1). It is necessary to select three consecutive WLs and a single WL during the majority and write operations respectively. A triple-row decoder [27] is used for this purpose. Assuming that every column of the array has a dedicated SA connected to it, the mapping is done in such a way that the latency is minimized. The majority gates to be operated in parallel during any step are mapped to different columns and same rows of the array. All the SAs connected to these columns produce the results in a single read cycle when these particular rows are selected using a triple-row decoder. The SA produces a majority gate's output and the corresponding complement at its output thereby avoiding the need for an extra inverter. The read-out outputs from SAs are stored in the registers and written into the array at appropriate cycles if the next subsequent operations requires them. Fig. 10

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:33:55 UTC from IEEE Xplore. Restrictions apply.

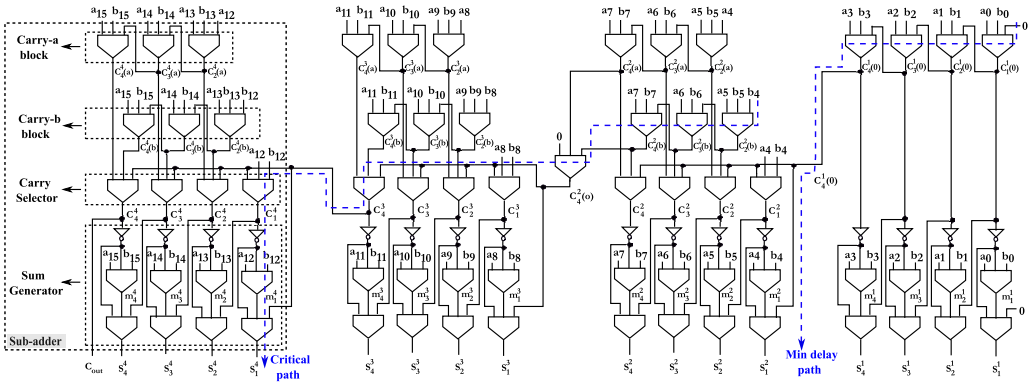


Fig. 9. Majority logic-based diagram of the proposed 16-bit SCSA-2 with sub-adder size $k = 4$. **Carry-a** and **Carry-b** modules in each sub-adder operate in parallel.

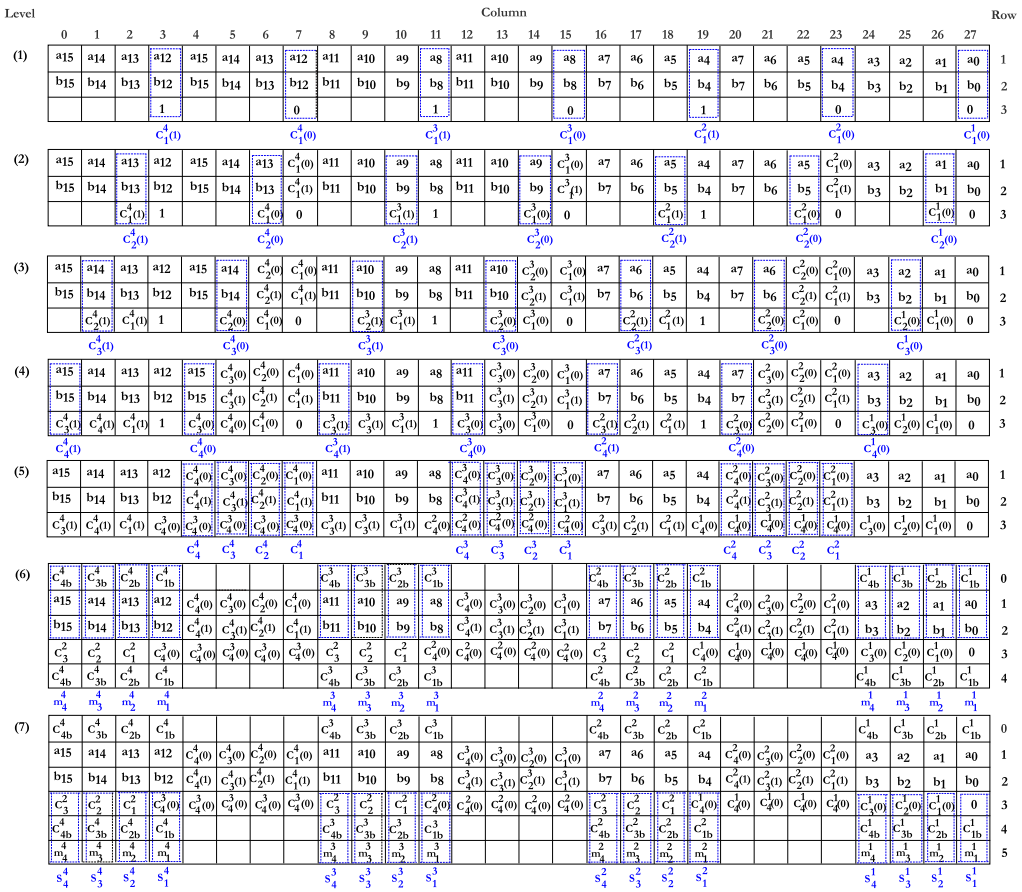


Fig. 10. Mapping of the proposed majority-based 16-bit SCSA-1 (Fig. 5) to a 6 × 28 1T–1R array. Each square box is assumed as a 1-bit 1T–1R cell. C^1_{1b} represents the complement of C^1_1 and both are produced at the sense amplifier output. The majority operations are shown with a dotted box and the result is shown below the corresponding column. The final result of a 16-bit approximate adder is $c^4_4 s^4_4 s^4_3 s^4_2 s^4_1 s^3_4 s^3_3 s^3_2 s^3_1 s^2_4 s^2_3 s^2_2 s^2_1 s^1_4 s^1_3 s^1_2 s^1_1$.

shows the mapping of the proposed 16-bit SCSA-1 (in Fig. 5) to a 1T–1R array and the contents of the array during the execution of each step. Detailed electrical simulation to verify the resilience of SA against process variations were carried out in [27]. Correct functionality of the proposed majority gate implemented as a READ operation using sense amplifier was verified by simulating with temperature variation and process corners using Cadence Spectre. A 1T-1R array consisting of a 45nm NMOS transistor in series with a ReRAM device as the memory cell was used for simulations. The Stanford-PKU model [43] which is a physics-based model was used to model the filamentary switching mechanism. The algorithm proposed in [27] was used to exactly fit the model to the characteristics of IHP's 1T–1R cell [44] (HRS = 200kΩ and LRS = 10.5kΩ, $V_{SET} = 0.8$V, $V_{RESET} = -1.1$V). Peripheral circuitry was designed in 45-nm CMOS technology. Table IV shows the ReRAM model parameters used for simulations. Our work in this manuscript is limited to simulations and layout implementation is not done. The energy for a single majority operation, E_{maj} = 0.7pJ and energy to write a

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:33:55 UTC from IEEE Xplore. Restrictions apply.

TABLE II

COMPARISON OF THE PROPOSED AND EXISTING IN-MEMORY APPROXIMATE ADDERS. THE REPORTED NMED, MRED AND LATENCY ARE FOR N/2 LEAST SIGNIFICANT INEXACT FULL ADDERS IN CASE OF IMPLY-BASED ADDERS IN [13]. THE LEAST REPORTED LATENCY AND THE CORRESPONDING ERROR METRICS ARE CONSIDERED FOR ADDERS IN [14] AND [38]

Adder type	Adder size		NMED ($\times 10^{-4}$)	MRED ($\times 10^{-4}$)	Latency (#cycles)	Energy (pJ)
IMPLY [13]	8		85	248	106	9988
	16		5.5	16	346	19980
	32		0.021	0.059	826	39950
IMPLY [14]	8		≈625	-	30	20-180
	16		≈61	-	60	40-360
	32		-	-	-	-
NOR [14]	8		≈703	-	40	1-5
	16		≈76	-	60	2-10
	32		-	-	-	-
MAJORITY [38]	8		800	-	17	-
	16		-	-	33	-
	32		-	-	65	-
Proposed SCSA-1	8	k=2	282	322	15	144.4
		k=4	0	0	23	114.12
	16	k=2	307	383	15	288.8
		k=4	18.8	25	23	228.2
		k=8	0	0	39	197.9
	32	k=2	309	379	15	577.6
		k=4	18.9	25	23	456.5
		k=8	0.08	0.1	39	395.9
		k=16	0	0	71	365.6
Proposed SCSA-2	8	k=2	62	87.7	17	160.2
		k=4	0	0	25	122
	16	k=2	76.7	102	17	320.5
		k=4	1.16	1.6	25	244.1
		k=8	0	0	41	205.9
	32	k=2	77.7	104	17	640.9
		k=4	1.2	1.7	25	488.2
		k=8	0.0003	0.0004	41	411.8
		k=16	0	0	73	373.6
Truncated adder [41]	8	k=2	46.87	-	31	78
		k=4	207	-	23	57.1
	16	k=2	-	-	63	161.9
		k=4	0.806	-	55	140.9
		k=8	13	-	39	98.9
	32	k=2	-	-	127	329.5
		k=4	-	-	119	308.6
		k=8	0.198	-	103	266.7
		k=16	0.051	-	71	182.8
Conventional SCSA [32]	8	k=2	282	322	23	312.1
		k=4	0	0	31	281.8
	16	k=2	307	383	23	624.2
		k=4	18.8	25	31	563.6
		k=8	0	0	47	533.3
	32	k=2	309	379	23	1,248.3
		k=4	18.9	25	31	1,127.2
		k=8	0.08	0.1	47	1,066.6
		k=16	0	0	79	1,036.3

- Source does not provide this information.

single bit to 1T-1R array, E_{write} = 3.26pJ. Energy for n-bit SCAS-1 with sub-adder size of k is $((k+3)E_{maj} + (3k+4)E_{write}) \times \lceil \frac{n}{k} \rceil$. Energy for n-bit SCAS-2 with sub-adder size of k is $((k+4)E_{maj} + (3k+5)E_{write}) \times \lceil \frac{n}{k} \rceil$. For a 16-bit adder with sub-adder size of 4, energy is 228.24pJ for SCSA-1 and 244.08pJ for SCSA-2. From Table II, proposed adders consume less energy compared to the adders reported in [13], [32], and IMPLY based adders in [14]. Adder design in [41] and NOR based adders in [14] consume less energy compared to the proposed adders but their error metrics and latency are significantly high compared to the proposed adders.

V. COMPLEXITY ANALYSIS AND COMPARISONS

In this Section, we compare the proposed approximate adders with the state-of-the-art in terms of latency, area, error characteristics and analyse the complexity.

A. Latency and Area

The proposed majority based n-bit SCAS-1 requires a latency of $(4k+7)$ cycles in-memory using a ripple carry method. Carry computation requires $(4k-3)$ cycles (k read and $3(k-1)$ write) and the sum computation requires 10 cycles (3 read and 7 write). For example, Fig. 10 shows mapping for a 16-bit adder in Fig. 5 which has sub-adder size k = 4 and each sub-adder has 4 majority levels assuming ripple carry method, which are Level (1)-(4) in Fig. 10. Each level requires one read and 3 write (for writing intermediate results required for next read) cycles except level (1) which is only read operation assuming that inputs already exist in the array, *i.e* carry computation requires k read + 3*(k-1) write = $(4k-3)$ cycles. And level (5)-(7) in Fig. 10 indicate sum computation (last 3 majority levels in Fig. 5) which requires 3 read + 7 write (to write intermediate results for next read) cycles. The sum computation requires 10 cycles irrespective of the adder size. Hence the total latency becomes $(4k-3)+10 = (4k+7)$ cycles for the proposed SCSA-1 with sub-adder size k. The number of ReRAM cells required for proposed SCSA-1 is $3k(3\lceil \frac{n}{k} \rceil - 1)$. This area can be reduced by $3\lceil \frac{n}{k} \rceil$ using the area-optimized design (in Fig. 6) for SCAS-1. The proposed n-bit SCSA-2 requires a latency of $(4k+9)$ cycles in-memory using a ripple carry method, which is 2 cycles more when compared to proposed SCSA-1. This is due to the extra majority gate added in the critical path (as explained in Section III-B) which needs one read and one write (to write its result back to the array as an input for next read) cycle when implemented in a 1T–1R array. If the prefix method is used for implementing the carry generators, the latency is $(4log_2(k)+6)$ cycles (n-bit adder requires $(4log_2(n)+6)$ cycles to implement in-memory [27]) for the proposed SCSA-1 and $(4log_2(k)+8)$ for SCSA-2. And area of both the designs increases by approximately $6\lceil \frac{n}{k} \rceil$ compared to the area used by the ripple carry method.

The proposed and existing in-memory adders are compared in terms of latency, NMED and MRED in Table II. From a quick glance at Table II, it may appear that the NMED and MRED of our proposed SCSA-1 is worse than IMPLY [13]. But with careful obervation, the reader will recognise that this is the case only for sub-adder size, $k = 2$. For any adder size, the proposed approximate adders outperform all other appromimate adders in terms of NMED, MRED and latency for sub-adder size $k = 4$ or more. Latency of the proposed adders of any length is constant for a given sub-adder size. For example, latency of the proposed 8,16 and 32 bit SCSA-1 (SCSA-2) is 15 (17) cycles for a sub-adder size of 2-bit ($k = 2$). This is not the case with other in-memory adders [13], [14], [38] and their latency grows with the size of the adder. For the fair comparison, ripple carry method is assumed to be used for carry generation in all the adders considered. For an n-bit RCA, the reported NMED, MRED and latency are for

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:33:55 UTC from IEEE Xplore. Restrictions apply.

TABLE III

COMPARISON OF THE PROPOSED AND EXISTING APPROXIMATE ADDERS OF SIZE 8, 16 AND 32-BIT FROM THE PERSPECTIVE OF ACCURACY. THE ERROR METRICS ARE CALCULATED WITH ALL POSSIBLE INPUT SAMPLES FOR 8-BIT AND 10 MILLION RANDOM SAMPLES FOR 16 AND 32-BIT ADDERS

Adder type	Sub-adder size	n = 8			n = 16			n = 32		
		ER (%)	NMED ($\times 10^{-4}$)	MRED ($\times 10^{-4}$)	ER (%)	NMED ($\times 10^{-4}$)	MRED ($\times 10^{-4}$)	ER (%)	NMED ($\times 10^{-4}$)	MRED ($\times 10^{-4}$)
BCSA [36]	2	27.47	395	428	60.98	417	554	87.62	417	554
	4	5.46	34	52	21.15	56	81	45.94	55	81
	8	0	0	0	6.2	2	3	18.04	3	4
BCSA-ERU [36]	2	18.73	167	185	47.86	172	190	74.66	172	189
	4	0	0	0	5.89	20	26	16.66	20	26
	8	0	0	0	0	0	0	0.39	0.1	0.07
HABA [35]	2	12.42	607	621	33.77	608	623	51.23	625	644
	4	1.37	137	136	4.29	155	157	9.18	156	158
	8	0	0	0	0.1	10	10	0.29	10	10
HABA-ERU [35]	2	12.41	151	155	33.77	154	157	51.21	156	161
	4	1.36	9	9	4.28	10	10	9.17	10	10
	8	0	0	0	0.1	0.04	0.04	0.29	0.04	0.04
DSLLA [37]	4	-	-	-	87.3	2.63	7.2	-	-	-
Proposed SCSA-1	2	17.8	282	322	48	307	383	78.6	309	379
	4	0	0	0	5.7	18.8	25	16.7	18.9	25
	8	0	0	0	0	0	0	0.38	0.08	0.1
Proposed SCSA-2	2	9.8	62	87.7	41.8	76.7	102	75.8	77.7	104
	4	0	0	0	2.98	1.16	1.6	13.9	1.2	1.7
	8	0	0	0	0	0	0	0.19	2.86×10^{-4}	3.96×10^{-4}
APPA [39]	NA	24.61	610	698	-	-	-	-	-	-

- Source does not provide this information. NA is not applicable.

TABLE IV

STANFORD-PKU RERAM MODEL PARAMETERS

$E_a = 0.6$ eV	$a_0 = 2.5 \times 10^{-10}$ m	$t_{ox} = 6 \times 10^{-9}$ m
$R_{TH} = 1500$ K/W	$I_0 = 6.8 \times 10^{-4}$ A	$g_0 = 0.3 \times 10^{-9}$ m
$\beta = 0.4, \gamma_0 = 20$	$gap_{min} = 0.85 \times 10^{-9}$ m	$\upsilon_0 = 0.4$ nm/ns
$T_{CRIT} = 450$ K	$gap_{max} = 1.8 \times 10^{-9}$ m	$\delta_g^0 = 0.005$ m
$T_0 = 298$ K	$T_{SMITH} = 500$ K	$V_0 = 0.35$ V

n/2 least significant inexact full adders in case of IMPLY-based adders in [13]. The least reported latency from the library of adders and the corresponding error metrics are considered for the adders in [14].

It is clear from Table II that the proposed adders has less latency and better error metrics compared to the existing designs. For a 16-bit adder with sub-adder size of 4-bit, the latency of the proposed SCSA-1 (SCSA-2) is 15 (13.8) times less than the latency of the adder reported in [13], 2.6 (2.4) times less than the latency of the adder reported in [14], 1.4 (1.3) times less compared to the latency required for in-memory implementation of majority based adders reported in [38]. We calculated latency for the conventional SCSA [32] by using the same method used for sub-adder in Fig. 5 (ripple carry method is assumed for sub-adder) of the proposed adder, where entire adder is realized in terms of majority gates and then mapped to a 1T–1R array (following the method in Fig. 10). Latency for the proposed SCSA-1 (SCSA-2) is 1.35 (1.24) times less and energy for the proposed SCSA-1 (SCSA-2) is 2.5 (2.3) times less compared to the conventional SCSA [32]. From Table II, proposed adders have less latency than the truncated adder [41]. In terms of accuracy, truncated adder is better for lesser k (no of truncated bits). As k grows, accuracy of the truncated adder degrades. For example of a 32 bit adder with k = 8, NMED = 0.198×10^{-4} for truncated adder and 0.0003×10^{-4} for our SCSA-2 adder, *i.e.* accuracy is improved by 99.84% with our adder. For the same example, we calculated latency for a truncated adder by using the same method used for sub-adder in Fig. 5 (ripple carry method is assumed for sub-adder) of the proposed adder, where entire adder is realized in terms of majority gates and then mapped to a 1T–1R array (following the method in Fig. 10). For n-bit adder with k bits truncated, carry computation requires (4(n-k)-3) cycles (n-k read and 3(n-k-1) write) and the sum computation requires 10 cycles, so latency is 4(n-k)+7 cycles. For a 32 bit adder (n = 32) with k = 8, latency is 103 cycles with truncated adder and 41 cycles for our SCSA-2 adder. The proposed adder shows significantly better accuracy and latency compared to truncated adder for large sized adders. Moreover, latency of the truncation-based and other adders grows with adder size but the proposed adders have same latency irrespective of adder size for a fixed sub-adder size.

B. Error Characteristics

Error characteristics such as ED, MED, ER, NMED, MSE, and MRED calculation is automated using python code for the proposed SCSA-1 and SCSA-2 by considering all possible input combinations for 8-bit adders, 10 million random input combinations for 16-bit and 32-bit adders. Table III shows the comparison of the proposed adders with the state-of-the-art approximate adders reported with best error characteristics,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:33:55 UTC from IEEE Xplore. Restrictions apply.

area and delay. Table III focuses solely on the error characteristics of the adder at the gate level and does not account for in-memory implementation. The comparison is done in terms of ER, NMED, and MRED for the adder sizes 8-bit, 16-bit, and 32-bit. From Table III, it is observed that the proposed SCSA-1 has less NMED, and MRED compared to the existing designs BCSA and HABA (without error recovery unit). Whereas the proposed SCSA-2 has less NMED, and MRED compared to the existing designs with and without an error recovery unit. The error metrics reduce with increase in sub-adder size for all the adders. The error rate of the proposed SCSA-2 is 51.28% less than the BCSA-ERU and 34.48% less than the HABA-ERU for a 32-bit adder with sub-adder size of 8-bit. The proposed SCSA-2 has 96.58% less error rate, 55.89% less NMED and 77.78% less MRED compared to the DSLLA [37] for a 16-bit adder. Moreover, the proposed SCSA-2 does not use any additional circuitry as used by the BCSA and HABA with error recovery unit (BCSA-ERU & HABA-ERU). The proposed SCSA-1 (SCSA-2) has 2.84 (12.9) times less NMED compared to the existing majority logic based approximate adder in the literature [38]. The worst possible error rate for an 8-bit proposed SCSA-1 (SCSA-2) is 17.8% (9.8%), which is less compared to the error rate reported in [39]. To conclude, the proposed SCSA-1 and SCSA-2 have inherently better error characteristics regardless of the physical implementation *i.e.* whether they are implemented in-memory or conventional CMOS logic.

VI. Conclusion

This work implements a speculative carry select adder in a 1T–1R array. The proposed majority logic based approximate adders completely eliminate the multiplexers in the critical path of a conventional speculative carry select adder. This improves the latency and the area overhead. The proposed adders are compared with the stat-of-the-art approximate adders implemented on FPGA and also in-memory. The error characteristics are independent of the physical implementation of the approximate adder - in CMOS or ReRAM-based memory with peripheral. In this paper, we have proposed two approximate adders (SCSA-1 and SCSA-2) whose error characteristics are inherently better (34.48% less error rate) than the state-of-the-art approximate adders in literature (Table III). Since the recent trend in research is to move computing to the memory, we implemented our approximate adders in memory and show that our adders are better than (4.3 times less latency) the in-memory approximate adders in literature (Table II). Approximate computing when combined with in-memory implementation can be an attractive option for image processing applications since image processing can be approximate (due to the limited perception of the human eye) and will benefit greatly from in memory implementation (due to large volume of image data whose transfer from off-chip memory is latency and energy intensive).

References

[1] M. Ali, S. Roy, U. Saxena, T. Sharma, A. Raghunathan, and K. Roy, "Compute-in-memory technologies and architectures for deep learning workloads," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 11, pp. 1615–1630, Nov. 2022.

[2] L. Han et al., "Efficient discrete temporal coding spike-driven in-memory computing macro for deep neural network based on nonvolatile memory," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 11, pp. 4487–4498, Nov. 2022.

[3] H. E. Yantır, A. M. Eltawil, and F. J. Kurdahi, "A hybrid approximate computing approach for associative in-memory processors," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 8, no. 4, pp. 758–769, Dec. 2018.

[4] O. Krestinskaya, L. Zhang, and K. N. Salama, "Towards efficient in-memory computing hardware for quantized neural networks: State-of-the-art, open challenges and perspectives," *IEEE Trans. Nanotechnol.*, vol. 22, pp. 377–386, 2023.

[5] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2014, pp. 10–14.

[6] M. Ahmadinejad and M. H. Moaiyeri, "Energy- and quality-efficient approximate multipliers for neural network and image processing applications," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 2, pp. 1105–1116, Apr. 2022.

[7] H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, "Approximate arithmetic circuits: A survey, characterization, and recent applications," *Proc. IEEE*, vol. 108, no. 12, pp. 2108–2135, Dec. 2020.

[8] Y. Kim, Y. Zhang, and P. Li, "Energy efficient approximate arithmetic for error resilient neuromorphic computing," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 11, pp. 2733–2737, Nov. 2015.

[9] F. Sabetzadeh, M. H. Moaiyeri, and M. Ahmadinejad, "A majority-based imprecise multiplier for ultra-efficient approximate image multiplication," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 11, pp. 4200–4208, Nov. 2019.

[10] W. Liu, C. Gu, M. O'Neill, G. Qu, P. Montuschi, and F. Lombardi, "Security in approximate computing and approximate computing for security: Challenges and opportunities," *Proc. IEEE*, vol. 108, no. 12, pp. 2214–2231, Dec. 2020.

[11] S. Raghuram and N. Shashank, "Approximate adders for deep neural network accelerators," in *Proc. 35th Int. Conf. VLSI Design 21st Int. Conf. Embedded Syst. (VLSID)*, Feb. 2022, pp. 210–215.

[12] W. Ahmad, B. Ayrancioglu, and I. Hamzaoglu, "Low error efficient approximate adders for FPGAs," *IEEE Access*, vol. 9, pp. 117232–117243, 2021.

[13] S. E. Fatemieh, M. R. Reshadinezhad, and N. TaheriNejad, "Fast and compact serial IMPLY-based approximate full adders applied in image processing," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 13, no. 1, pp. 175–188, Mar. 2023.

[14] C. K. Jha, P. L. Thangkhiew, K. Datta, and R. Drechsler, "Imagin: Library of imply and MAGIC NOR-based approximate adders for in-memory computing," *IEEE J. Explor. Solid-State Computat. Devices Circuits*, vol. 8, no. 2, pp. 68–76, Dec. 2022.

[15] S. Muthulakshmi, C. S. Dash, and S. R. S. Prabaharan, "Memristor augmented approximate adders and subtractors for image processing applications: An approach," *AEU Int. J. Electron. Commun.*, vol. 91, pp. 91–102, Jul. 2018.

[16] S. Kvatinsky, N. Wald, G. Satat, A. Kolodny, U. C. Weiser, and E. G. Friedman, "MRL—Memristor ratioed logic," in *Proc. 13th Int. Workshop Cellular Nanosc. Netw. Appl.*, Aug. 2012, pp. 1–6.

[17] T. Zhang et al., "A survey of majority logic designs in emerging nanotechnologies for computing," *IEEE Trans. Nanotechnol.*, vol. 22, pp. 732–739, 2023.

[18] K. Kong, Y. Shang, and R. Lu, "An optimized majority logic synthesis methodology for quantum-dot cellular automata," *IEEE Trans. Nanotechnol.*, vol. 9, no. 2, pp. 170–183, Mar. 2010.

[19] J. Reuben and S. Pechmann, "A parallel-friendly majority gate to accelerate in-memory computation," in *Proc. IEEE 31st Int. Conf. Application-Specific Syst., Architectures Processors (ASAP)*, Jul. 2020, pp. 93–100.

[20] S. G. Rohani, N. Taherinejad, and D. Radakovits, "A semiparallel full-adder in IMPLY logic," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 1, pp. 297–301, Jan. 2020.

[21] R. Ben-Hur et al., "SIMPLER MAGIC: Synthesis and mapping of in-memory logic executed in a single row to improve throughput," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 10, pp. 2434–2447, Oct. 2020.

[22] V. Lakshmi, J. Reuben, and V. Pudi, "A novel in-memory Wallace tree multiplier architecture using majority logic," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 3, pp. 1148–1158, Mar. 2022.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:33:55 UTC from IEEE Xplore. Restrictions apply.

[23] N. Burgess, "Prenormalization rounding in IEEE floating-point operations using a flagged prefix adder," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 13, no. 2, pp. 266–277, Feb. 2005.
[24] C. Liu, J. Han, and F. Lombardi, "An analytical framework for evaluating the error characteristics of approximate adders," *IEEE Trans. Comput.*, vol. 64, no. 5, pp. 1268–1281, May 2015.
[25] S. Yu, "Resistive random access memory (RRAM)," *Synthesis Lect. Emerg. Eng. Technol.*, vol. 2, no. 5, pp. 1–79, 2016.
[26] J. Reuben, D. Fey, and C. Wenger, "A modeling methodology for resistive RAM based on stanford-PKU model with extended multilevel capability," *IEEE Trans. Nanotechnol.*, vol. 18, pp. 647–656, 2019.
[27] J. Reuben and S. Pechmann, "Accelerated addition in resistive RAM array using parallel-friendly majority gates," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 29, no. 6, pp. 1108–1121, Jun. 2021.
[28] V. Lakshmi, V. Pudi, and J. Reuben, "Inner product computation in-memory using distributed arithmetic," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 11, pp. 4546–4557, Nov. 2022.
[29] V. Lakshmi, J. Reuben, and V. Pudi, "Majority logic based in-memory comparator," in *Proc. IEEE Int. Conf. Semiconductor Electron. (ICSE)*, Aug. 2022, pp. 105–108.
[30] L. Brackmann, T. Ziegler, D. J. Wouters, and S. Menzel, "Experimental verification and evaluation of non-stateful logic gates in resistive RAM," *IEEE Trans. Circuits Syst. I, Reg. Papers*, early access, Nov. 1, 2024, doi: 10.1109/TCSI.2024.3486376.
[31] H. Jiang, J. Han, and F. Lombardi, "A comparative review and evaluation of approximate adders," in *Proc. 25th Great Lakes Symp. VLSI*, 2015, pp. 343–348.
[32] K. Du, P. Varman, and K. Mohanram, "High performance reliable variable latency carry select addition," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2012, pp. 1257–1262.
[33] I.-C. Lin, Y.-M. Yang, and C.-C. Lin, "High-performance low-power carry speculative addition with variable latency," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 9, pp. 1591–1603, Sep. 2015.
[34] J. M. Rabaey, *Digital Integrated Circuits a Design Perspective*. Upper Saddle River, NJ, USA: Prentice-Hall, 1999.
[35] J. Hu and W. Qian, "A new approximate adder with low relative error and correct sign calculation," in *Proc. Design, Autom. Test Europe Conf. Exhib.*, 2015, pp. 1449–1454.
[36] F. Ebrahimi-Azandaryani, O. Akbari, M. Kamal, A. Afzali-Kusha, and M. Pedram, "Block-based carry speculative approximate adder for energy-efficient applications," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 67, no. 1, pp. 137–141, Jan. 2020.
[37] H. Seo and Y. Kim, "A low latency approximate adder design based on dual sub-adders with error recovery," *IEEE Trans. Emerg. Topics Comput.*, vol. 11, no. 3, pp. 811–816, Jul./Sep. 2023.
[38] W. Liu, T. Zhang, E. McLarnon, M. O'Neill, P. Montuschi, and F. Lombardi, "Design and analysis of majority logic-based approximate adders and multipliers," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 3, pp. 1609–1624, Jul./Sep. 2021.
[39] B. Rashidi, "APPAs: Fast and efficient approximate parallel prefix adders and multipliers," *J. Supercomput.*, vol. 80, no. 16, pp. 24269–24296, Nov. 2024.
[40] B. Rashidi, "Efficient and low-cost approximate multipliers for image processing applications," *Integration*, vol. 94, Jan. 2024, Art. no. 102084.
[41] F. Frustaci, S. Perri, P. Corsonello, and M. Alioto, "Energy-quality scalable adders based on nonzeroing bit truncation," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 4, pp. 964–968, Apr. 2019.
[42] V. Pudi, K. Sridharan, and F. Lombardi, "Majority logic formulations for parallel adder designs at reduced delay and circuit complexity," *IEEE Trans. Comput.*, vol. 66, no. 10, pp. 1824–1830, Oct. 2017.
[43] Z. Jiang et al., "A compact model for metal-oxide resistive random access memory with experiment verification," *IEEE Trans. Electron Devices*, vol. 63, no. 5, pp. 1884–1892, May 2016.
[44] S. Pechmann et al., "A versatile, voltage-pulse based read and programming circuit for multi-level RRAM cells," *Electronics*, vol. 10, no. 5, p. 530, Feb. 2021.



Vijaya Lakshmi received the B.Tech. degree in electronics and communication engineering from Madanapalle Institute of Technology, Madanapalle, India, in 2014, and the master's degree in VLSI design from the Government College of Technology, Coimbatore, India, in 2016. She is currently pursuing the Ph.D. degree with Indian Institute of Technology Tirupati, India. She was an Assistant Professor with the Karpagam College of Engineering, Coimbatore, for three years. Her research interests include in-memory computing using emerging non-volatile memories.



Vikramkumar Pudi (Member, IEEE) received the Ph.D. degree from Indian Institute of Technology Madras in 2014. He was a Post-Doctoral Fellow with the Microsystems and Nanotechnology Group, The University of British Columbia, Canada, from 2014 to 2015. From 2015 to 2018, he was with the School of Computer Science and Engineering, Nanyang Technological University, Singapore, as a Research Fellow. He is currently an Assistant Professor with Indian Institute of Technology Tirupati. He is the author of two Springer books on quantum dot cellular automata and carbon nanotube field effect transistor technology. His research interests include digital design, VLSI architectures, in-memory computing, and nanoelectronics.



John Reuben received the B.E. degree (Hons.) from the Birla Institute of Technology and Science, Pilani (BITS Pilani), in 2004, and the master's and Ph.D. degree from VIT University, Vellore, India, in 2008 and 2015, respectively. From January 2017 to January 2018, he was a Post-Doctoral Researcher at the Technion—Israel Institute of Technology. He is currently a Post-Doctoral Researcher at Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU). His research interests include emerging non-volatile memories (resistive RAMs and ferroelectric devices) and their applications to in-memory/near-memory computing. He was a recipient of the Viterbi Fellowship (Technion, Israel) and Emerging Talents Initiative (FAU, Germany).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:33:55 UTC from IEEE Xplore. Restrictions apply.